UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME TÉCNICO – PROYECTO DEL PRIMER PARCIAL

SISTEMA DE GESTIÓN DE RUTAS ÓPTIMAS EN CUSCO: ANÁLISIS COMPARATIVO DE ALGORITMOS GREEDY, DIVIDE Y VENCERÁS, PROGRAMACIÓN DINÁMICA Y BACKTRACKING

Curso:
PROGRAMACIÓN III

Integrantes:
Amilcar Estacio Medrano (200822)
Kevin Heli Choquecahua Gutierrez (245570)
Jeliel Enrique Aramburu (210927)

Docente:
Mg. Karla Catherine Rojas Pedraza

Perú, Octubre de 2026

Lista de Acrónimos

APA	American Psychological Association
B&B	Branch and Bound (Ramificación y Poda)
CVRP	Capacitated Vehicle Routing Problem (Problema de Enrutamiento de Vehículos con Capacidad)
DFS	Depth-First Search (Búsqueda en Profundidad)
DyV	Divide y Vencerás (Divide and Conquer)
EPIIS	Escuela Profesional de Ingeniería Informática y de Sistemas
GPS	Global Positioning System (Sistema de Posicionamiento Global)
GUI	Graphical User Interface (Interfaz Gráfica de Usuario)
JSON	JavaScript Object Notation (Notación de Objetos de JavaScript)
OSM	OpenStreetMap
PD	Programación Dinámica (Dynamic Programming)
TSP	Travelling Salesperson Problem (Problema del Viajante de Comercio)
UNSAAC	Universidad Nacional de San Antonio Abad del Cusco
VRP	Vehicle Routing Problem (Problema de Enrutamiento de Vehículos)
WGS84	World Geodetic System 1984 (Sistema Geodésico Mundial 1984)

Agradecimientos

Agradecemos a la Universidad Nacional de San Antonio Abad del Cusco (UNSAAC) y a la Escuela Profesional de Ingeniería Informática y de Sistemas por brindarnos la formación académica y científica necesaria para el desarrollo de este proyecto.

Expresamos nuestra sincera gratitud a la docente del curso de Programación III, Mg. Karla Catherine Rojas Pedraza, por sus valiosas orientaciones pedagógicas, exigencia metodológica y retroalimentación constante durante el semestre académico. Asimismo, extendemos nuestro reconocimiento a nuestros compañeros de equipo por el compromiso y esfuerzo compartido a lo largo de cada fase de este trabajo.

Índice general

Lista de Acrónimos **II**

Agradecimientos **III**

Índice de Tablas **VII**

Índice de Figuras **VIII**

Resumen **IX**

Abstract **X**

1. Introducción **1**

2. Planteamiento del Problema **3**

2.1. Descripción de la Situación Problemática 3

2.2. Problema General 4

2.3. Problemas Específicos 4

2.4. Justificación 4

3. Objetivos de la Investigación **6**

3.1. Objetivo General 6

3.2. Objetivos Específicos 6

4. Marco Teórico **7**

4.1. Antecedentes 7

4.2. Bases Teóricas 8

4.2.1. Grafos y Representación de la Red Vial 8
4.2.2. Paradigma Greedy 8
4.2.3. Paradigma Divide y Vencerás 9
4.2.4. Programación Dinámica 9
4.2.5. Backtracking 10
4.2.6. Notación Big-O 10

5. Hipótesis de Investigación **11**
5.1. Hipótesis General 11
5.2. Hipótesis Específicas 11
5.3. Verificación de Hipótesis 12

6. Metodología de la Investigación **13**
6.1. Tipo de Investigación 13
6.2. Proceso Metodológico 13
6.2.1. Fase 1: Diseño de la Arquitectura 13
6.2.2. Fase 2: Implementación Algorítmica 14
6.2.3. Fase 3: Construcción de la Interfaz Gráfica 15
6.2.4. Fase 4: Verificación y Benchmarking 15
6.3. Instrumentos y Entorno de Desarrollo 15

7. Resultados y Discusión **17**
7.1. Resultados 17
7.1.1. Análisis de Complejidad Teórica 17
7.1.2. Benchmarking Empírico: Tiempo de Ejecución 17
7.1.3. Calidad de Solución: Distancia Total de Ruta 18
7.1.4. Backtracking: Rutas Alternativas con Poda 18
7.1.5. Verificación de Correctitud: Pruebas Unitarias 19
7.2. Discusión 19

8. Conclusiones y Recomendaciones **22**

8.1. Conclusiones 22
8.2. Recomendaciones 23

Referencias **24**

Apéndice y Anexos **25**

Apéndice y Anexos **27**

Índice de Tablas

7.1. Complejidad Big-O de los algoritmos implementados. 17

7.2. Tiempos de ejecución promedio en ms — Greedy vs Held-Karp (datos reales). 18

7.3. Distancia total de ruta (m) y gap de suboptimalidad del Greedy respecto a Held-Karp. 19

7.4. Resultados del Backtracking en cuatro escenarios del grafo de Cusco. . . 20

8.1. Nodos de referencia geográfica del grafo simulado de Cusco. 27

Índice de Figuras

7.1. Tiempo de ejecución Greedy vs Held-Karp en escala logarítmica 18

7.2. Comparación de calidad de ruta: Greedy vs Held-Karp 19

7.3. Rendimiento del Backtracking con poda 20

Resumen

El presente trabajo desarrolla un sistema de gestión de rutas óptimas para reparto de pedidos en la ciudad del Cusco, implementado como aplicación de escritorio en Python con interfaz gráfica basada en Tkinter. El problema central consiste en determinar, de forma eficiente, qué pedidos debe atender un repartidor y en qué orden visitarlos, minimizando la distancia total recorrida y respetando restricciones de capacidad del vehículo y calles cerradas en la red vial. Para resolverlo se implementaron, desde cero y sin librerías de optimización externas, cuatro paradigmas algorítmicos: (1) un algoritmo *Greedy* de vecino más cercano para construcción de ruta; (2) *Merge Sort* (Divide y Vencerás) para ordenamiento de pedidos; (3) *Knapsack 0/1* y *Held-Karp* (Programación Dinámica) para selección óptima de pedidos y ruta TSP exacta; y (4) *Backtracking* con poda de ciclos para exploración de rutas alternativas con calles bloqueadas. Los *benchmarks* empíricos confirmaron la complejidad teórica: Held-Karp alcanzó 42.2 ms para $n = 12$ pedidos con un speedup del Greedy de $138{,}8\times$, mientras que el gap de suboptimalidad del Greedy creció de $0\,\%$ en $n = 3$ a $14.81\,\%$ en $n = 12$. El Backtracking encontró entre 3 y 9 rutas alternativas en menos de 1.1 ms en cuatro escenarios del grafo de Cusco. El sistema resultante es estable, visualmente verificable y permite reproducir todos los escenarios de prueba de forma interactiva.

Palabras clave: rutas óptimas, algoritmos greedy, programación dinámica, backtracking, Held-Karp, Cusco, TSP, Knapsack

Abstract

This work presents an optimal route management system for parcel delivery in the city of Cusco, implemented as a Python desktop application with a Tkinter-based graphical user interface. The core problem consists of efficiently determining which delivery orders a courier should fulfill and the sequence in which to visit them, minimizing total travel distance while respecting vehicle capacity constraints and the existence of blocked streets in the road graph. To solve it, four algorithmic paradigms were implemented from scratch without external optimization libraries: (1) a *Greedy* nearest-neighbor heuristic for route construction; (2) *Merge Sort* (Divide and Conquer) for order sorting; (3) *0/1 Knapsack* and *Held-Karp* (Dynamic Programming) for optimal order selection and exact TSP routing; and (4) *Backtracking* with cycle pruning for alternative route exploration when streets are blocked. Empirical benchmarks confirmed the theoretical complexity predictions: Held-Karp reached 42.2 ms for $n = 12$ orders with a Greedy speedup of $138{,}8\times$, while the Greedy suboptimality gap grew from $0\,\%$ at $n = 3$ to $14.81\,\%$ at $n = 12$. The Backtracking algorithm found between 3 and 9 alternative routes in less than 1.1 ms across four Cusco graph scenarios. The resulting application is stable, visually verifiable, and allows all test scenarios to be reproduced interactively.

Keywords: optimal routes, greedy algorithms, dynamic programming, backtracking, Held-Karp, Cusco, TSP, Knapsack

1. Introducción

El problema de la distribución eficiente de bienes en entornos urbanos es uno de los desafíos logísticos más relevantes tanto en la industria como en el campo del diseño algorítmico. En ciudades como el Cusco, cuya red vial histórica presenta calles angostas, sentidos únicos, zonas peatonales y calles frecuentemente bloqueadas por actividades culturales o de mantenimiento, la planificación manual de rutas de reparto resulta especialmente compleja e ineficiente.

El presente proyecto, desarrollado en el marco del curso de Programación III de la Escuela Profesional de Ingeniería Informática y de Sistemas (EPIIS) de la Universidad Nacional de San Antonio Abad del Cusco (UNSAAC), aborda esta problemática mediante la construcción de un sistema de gestión de rutas óptimas implementado como aplicación de escritorio en Python. El sistema integra una interfaz gráfica de usuario (GUI) basada en Tkinter que permite visualizar el grafo vial simulado de Cusco, gestionar pedidos de reparto y ejecutar cuatro paradigmas algorítmicos fundamentales del diseño de algoritmos: *Greedy*, *Divide y Vencerás*, *Programación Dinámica* y *Backtracking*.

El trabajo no se limita a la implementación funcional de los algoritmos, sino que incluye un análisis riguroso de su complejidad computacional temporal y espacial en notación Big-O, así como la ejecución de *benchmarks* empíricos que permiten comparar el rendimiento real de cada paradigma ante diferentes volúmenes de datos. De este modo, el sistema sirve simultáneamente como herramienta práctica de logística urbana simulada y como plataforma de demostración pedagógica de los paradigmas algorítmicos estudiados durante el semestre.

El documento se organiza de la siguiente manera: el Capítulo 2 describe el planteamiento del problema y su justificación; el Capítulo 3 establece los objetivos del proyecto;

el Capítulo 4 desarrolla el marco teórico de los algoritmos implementados; el Capítulo 5 formula la hipótesis de trabajo; el Capítulo 6 describe la metodología de desarrollo; el Capítulo 7 presenta los resultados obtenidos y su discusión comparativa; y el Capítulo 8 expone las conclusiones y recomendaciones finales.

2. Planteamiento del Problema

2.1 Descripción de la Situación Problemática

La ciudad del Cusco posee una red vial de carácter histórico que impone restricciones singulares a la movilidad urbana. Calles con ancho insuficiente para vehículos de carga, zonas de tráfico restringido en el Centro Histórico, sentidos únicos de circulación, y cierres temporales frecuentes por eventos culturales o trabajos de mantenimiento, hacen que la planificación manual de rutas de distribución sea altamente ineficiente. A esto se suma la creciente demanda de servicios de reparto a domicilio, que ha crecido de manera sostenida en los últimos años tanto en el ámbito nacional como en la región del Cusco.

Las empresas y emprendimientos locales que realizan reparto de pedidos —restaurantes, farmacias, tiendas de abarrotes, comercio electrónico local— carecen, en su mayoría, de herramientas computacionales que les permitan determinar de forma sistemática qué pedidos priorizar cuando la capacidad del vehículo es limitada, y en qué orden visitarlos para minimizar el tiempo y la distancia recorrida. En la práctica, esta decisión se delega al criterio empírico del repartidor, lo que conduce a rutas subóptimas, mayor consumo de combustible, entregas tardías y reducción de la capacidad de atención diaria.

Desde el punto de vista algorítmico, el problema de la selección óptima de pedidos bajo restricción de capacidad corresponde al problema de la Mochila 0/1 (*Knapsack 0/1*), mientras que la determinación del orden de visita óptimo es una instancia del Problema del Viajante de Comercio (*Travelling Salesperson Problem*, TSP). Ambos pertenecen a la clase de problemas NP-difíciles para los cuales la Programación Dinámica ofrece soluciones exactas de complejidad exponencial, y las heurísticas *Greedy* constituyen alternativas prácticas de complejidad polinomial. La incorporación de restricciones de calles bloqueadas introduce la necesidad de técnicas de búsqueda exhaustiva como el

Backtracking, mientras que la organización y preprocesamiento de los pedidos se beneficia del paradigma *Divide y Vencerás*.

Este proyecto responde a la necesidad académica de contrastar empíricamente estos cuatro paradigmas sobre un caso de uso concreto y relevante localmente, analizando sus diferencias de rendimiento y calidad de solución.

2.2 Problema General

¿En qué medida el diseño e implementación de un sistema computacional que aplica los paradigmas algorítmicos Greedy, Divide y Vencerás, Programación Dinámica y Backtracking permite gestionar eficientemente las rutas de reparto en la ciudad del Cusco, optimizando la distancia total recorrida y el cumplimiento de restricciones operativas?

2.3 Problemas Específicos

El estudio toma en cuenta los siguientes problemas específicos:

- ¿Cómo afecta la elección del paradigma algorítmico (Greedy vs. Programación Dinámica) a la calidad de la selección de pedidos bajo restricción de capacidad, medida como distancia total de la ruta generada?
- ¿Cuál es la complejidad temporal y espacial real, medida empíricamente, de cada uno de los cuatro algoritmos implementados en función del número de pedidos activos?
- ¿En qué condiciones del grafo vial (número de aristas bloqueadas, densidad de la red) el algoritmo de Backtracking logra encontrar rutas alternativas viables en tiempo aceptable para un usuario interactivo?

2.4 Justificación

La justificación de este trabajo es de naturaleza académica y práctica. En el plano académico, el proyecto constituye una demostración integral de cuatro de los paradigmas

algorítmicos más importantes del diseño de algoritmos, aplicados sobre un problema de optimización combinatoria real. La implementación desde cero de cada algoritmo —sin recurrir a librerías de optimización externas— permite a los integrantes del equipo comprender en profundidad la estructura y el comportamiento computacional de cada paradigma, lo que resulta esencial para la formación en Ingeniería Informática y de Sistemas.

En el plano práctico, el sistema desarrollado demuestra que es posible construir, con recursos computacionales modestos y en Python, una herramienta de apoyo a la decisión logística para pequeñas y medianas empresas de reparto en Cusco. Si bien el grafo vial utilizado es una simulación con 20 nodos de referencia geográfica real, la arquitectura modular del sistema permite su extensión futura con datos reales obtenidos de OpenStreetMap.

La comparación cuantitativa entre algoritmos, respaldada por *benchmarks* empíricos y análisis Big-O, añade valor pedagógico al permitir visualizar de forma directa la diferencia entre complejidad polinomial y exponencial en un contexto aplicado.

3. Objetivos de la Investigación

3.1 Objetivo General

Diseñar e implementar un sistema de escritorio en Python que gestione pedidos de reparto y genere rutas óptimas en un grafo vial simulado de la ciudad del Cusco, integrando y comparando cuatro paradigmas algorítmicos —Greedy, Divide y Vencerás, Programación Dinámica y Backtracking— mediante análisis de complejidad Big-O y benchmarks empíricos, con el fin de determinar el paradigma más adecuado según el tamaño del problema y las restricciones operativas.

3.2 Objetivos Específicos

- Implementar desde cero los algoritmos Greedy de selección por densidad, Merge Sort (Divide y Vencerás), Knapsack 0/1 y Held-Karp (Programación Dinámica), y Backtracking con poda de calles bloqueadas, verificando su correctitud mediante pruebas unitarias con `pytest`.

- Derivar y documentar la complejidad temporal y espacial Big-O de cada algoritmo implementado, comparando el crecimiento teórico con los tiempos de ejecución medidos experimentalmente para entre 3 y 15 pedidos activos.

- Desarrollar una interfaz gráfica de usuario (GUI) con Tkinter que permita visualizar el grafo vial, gestionar pedidos y repartidores, ejecutar cada algoritmo de forma interactiva, y presentar los resultados de comparación en gráficos de rendimiento.

4. Marco Teórico

En este capítulo se presentan los fundamentos teóricos que sustentan el diseño e implementación del sistema. Se abordan los conceptos de grafos, los paradigmas algorítmicos implementados, los problemas de optimización combinatoria subyacentes y la notación de complejidad computacional empleada para el análisis.

4.1 Antecedentes

Dantzig and Ramser (1959) formularon el Problema de Enrutamiento de Vehículos (VRP) como generalización del TSP con múltiples vehículos y capacidad limitada, estableciendo las bases formales del problema que este proyecto aborda en escala reducida. Su trabajo demostró que la planificación óptima de rutas es NP-difícil en el caso general, lo que motiva el uso de heurísticas y algoritmos exactos de complejidad exponencial acotada.

Held and Karp (1962) propusieron el algoritmo de Programación Dinámica para el TSP que lleva su nombre (Held-Karp), reduciendo la complejidad de la fuerza bruta $O(n!)$ a $O(2^n \cdot n^2)$ mediante la técnica de memoización sobre subconjuntos de nodos visitados. Este es el algoritmo exacto implementado en el presente proyecto para la optimización de rutas de reparto.

Rosenkrantz et al. (1977) analizaron heurísticas constructivas para el TSP, incluyendo la heurística del vecino más cercano (*nearest neighbor*), que constituye la base del enfoque Greedy implementado en este trabajo. Sus resultados establecen que esta heurística produce rutas de a lo sumo el doble de la longitud óptima en el peor caso.

Cormen et al. (2022) proporcionan el tratamiento algorítmico formal de los paradigmas implementados: Divide y Vencerás (Merge Sort, capítulo 2), Programación Diná-

mica (Knapsack 0/1, capítulo 16) y Backtracking (capítulo 34), así como el análisis de complejidad Big-O utilizado en este informe.

Toth and Vigo (2014) presentan una revisión exhaustiva de métodos exactos y heurísticos para el VRP con restricciones de capacidad (CVRP), contextualizando el problema de selección y secuenciación de pedidos en el marco de la logística moderna.

4.2 Bases Teóricas

4.2.1. Grafos y Representación de la Red Vial

Un grafo $G = (V, E)$ consiste en un conjunto de vértices V (nodos) y un conjunto de aristas $E \subseteq V \times V$. En este proyecto, cada nodo $v_i \in V$ representa una ubicación georreferenciada en Cusco, caracterizada por una latitud ϕ_i y longitud λ_i. Cada arista $e_{ij} = (v_i, v_j, w_{ij}) \in E$ representa una conexión vial entre dos nodos con peso w_{ij} igual a la distancia ortodrómica calculada mediante la fórmula de Haversine:

$$d(i,j) = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\phi_j - \phi_i}{2}\right) + \cos\phi_i \cos\phi_j \sin^2\left(\frac{\lambda_j - \lambda_i}{2}\right)}\right) \tag{4.1}$$

donde $R = 6{,}371{,}000$ m es el radio medio de la Tierra (Cormen et al., 2022). El grafo se almacena como diccionario de listas de adyacencia, estructura que permite recorrer los vecinos de un nodo en $O(\deg(v))$ y ocupa $O(|V| + |E|)$ de espacio.

4.2.2. Paradigma Greedy

Un algoritmo *Greedy* construye una solución paso a paso, tomando en cada etapa la decisión localmente óptima sin reconsiderar elecciones anteriores (Cormen et al., 2022). En este proyecto se aplica para la selección de pedidos: se calcula para cada pedido p_i su densidad $\delta_i = \text{prioridad}_i / (\text{peso}_i \cdot d_i)$, donde d_i es la distancia desde el depósito hasta el nodo del pedido. Los pedidos se ordenan de mayor a menor δ_i y se seleccionan en ese orden mientras la capacidad restante del repartidor lo permita. Este proceso tiene complejidad $O(n \log n)$ por el ordenamiento y $O(n)$ para la selección, resultando en $O(n \log n)$ total.

4.2.3. Paradigma Divide y Vencerás

El paradigma *Divide y Vencerás* divide el problema en subproblemas del mismo tipo, los resuelve recursivamente y combina sus soluciones (Cormen et al., 2022). En este proyecto se implementa mediante el algoritmo **Merge Sort** para ordenar la lista de pedidos por distancia al depósito:

Algorithm 1: Merge Sort para pedidos (Divide y Vencerás)

Data: Lista A de n pedidos con distancias
Result: Lista A ordenada de menor a mayor distancia
if $|A| \leq 1$ **then**
  return A
end
mitad $\leftarrow \lfloor |A|/2 \rfloor$;
$L \leftarrow$ `merge_sort`$(A[:\text{mitad}])$;
$R \leftarrow$ `merge_sort`$(A[\text{mitad}:])$;
return `merge`(L, R);

La relación de recurrencia del Merge Sort es $T(n) = 2T(n/2) + O(n)$, cuya solución por el Teorema Maestro es $T(n) = O(n \log n)$, con espacio auxiliar $O(n)$.

4.2.4. Programación Dinámica

La *Programación Dinámica* (PD) resuelve un problema descomponiéndolo en subproblemas superpuestos cuya solución se almacena (*memoización* o tabulación) para evitar recálculos (Bellman, 1957).

Knapsack 0/1. Dado un conjunto de pedidos $\{p_1, \ldots, p_n\}$, cada uno con peso w_i y valor v_i, y una capacidad máxima W, se busca el subconjunto de pedidos que maximice $\sum v_i$ sujeto a $\sum w_i \leq W$. La tabla de PD $dp[i][c]$ representa el valor máximo considerando los primeros i pedidos con capacidad c:

$$dp[i][c] = \max\big(dp[i-1][c],\ v_i + dp[i-1][c-w_i]\big) \quad \text{si } w_i \leq c \tag{4.2}$$

Complejidad: $O(n \cdot W)$ tiempo y $O(n \cdot W)$ espacio (optimizable a $O(W)$ usando una sola fila).

Held-Karp (TSP exacto). Dado un conjunto de nodos de entrega $S = \{s_1, \ldots, s_n\}$ y las distancias $d(i, j)$ entre ellos, el algoritmo Held-Karp calcula la ruta hamiltoniana de mínimo costo que parte del depósito, visita todos los nodos de S exactamente una vez y

regresa al depósito (Held and Karp, 1962). Define:

$$C(S,i) = \min_{j \in S \setminus \{i\}} \left[C(S \setminus \{i\}, j) + d(j,i) \right] \tag{4.3}$$

donde $C(\{i\},i) = d(\text{depósito},i)$. El estado (S,i) codifica que se han visitado exactamente los nodos de S y el último visitado fue i. Con n nodos de entrega hay 2^n subconjuntos y n posibles últimos nodos, dando $O(2^n \cdot n)$ estados y $O(n)$ transiciones por estado: complejidad total $O(2^n \cdot n^2)$ tiempo y $O(2^n \cdot n)$ espacio.

4.2.5. Backtracking

El *Backtracking* es una técnica de búsqueda exhaustiva con poda: construye la solución incrementalmente y abandona una rama de la búsqueda (*retrocede*) tan pronto como detecta que no puede conducir a una solución válida (Cormen et al., 2022). En este proyecto se usa para encontrar rutas en el grafo vial que eviten aristas bloqueadas: dado un nodo origen y un nodo destino, el algoritmo explora en profundidad (DFS) todos los caminos posibles, marcando nodos visitados para evitar ciclos y podando si se supera una longitud máxima configurable. En el peor caso, sin bloqueos, el algoritmo recorre todos los caminos simples en el grafo, cuya cantidad es $O(n!)$; en la práctica, la poda y los bloqueos reducen dramáticamente el espacio de búsqueda.

4.2.6. Notación Big-O

La notación $O(f(n))$ describe el crecimiento asintótico del tiempo o espacio de un algoritmo en el peor caso (Cormen et al., 2022). Se dice que $T(n) = O(f(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$. Las complejidades relevantes para este proyecto, ordenadas de mejor a peor, son: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n \cdot W)$ (pseudopolinomial), $O(2^n \cdot n^2)$, $O(n!)$.

5. Hipótesis de Investigación

5.1 Hipótesis General

La implementación de un sistema computacional que integra los paradigmas algorítmicos Greedy, Divide y Vencerás, Programación Dinámica y Backtracking permite gestionar rutas de reparto en la ciudad del Cusco de forma sistemática y eficiente, con la condición de que el número de pedidos activos no supere 12 para que el algoritmo exacto Held-Karp sea ejecutable en tiempo interactivo (menor a 50 ms en hardware de escritorio estándar).

5.2 Hipótesis Específicas

- El algoritmo de Programación Dinámica Held-Karp producirá, en todos los casos evaluados, una distancia de ruta menor o igual a la del Greedy, con un gap de suboptimalidad del Greedy que crecerá con n, superando el 10 % a partir de $n \geq 8$ pedidos activos.

- El tiempo de ejecución medido empíricamente del algoritmo Held-Karp crecerá de forma exponencial con el número de pedidos n, mientras que el tiempo del algoritmo Greedy crecerá de forma polinomial, verificando los órdenes teóricos $O(2^n \cdot n^2)$ y $O(n^2)$, respectivamente.

- El algoritmo de Backtracking encontrará todas las rutas alternativas simples entre el nodo origen y destino en el grafo vial, aplicando poda de ciclos eficientemente, con un tiempo de ejecución menor a 2 ms para los pares de nodos evaluados en el grafo de 20 nodos de Cusco.

5.3 Verificación de Hipótesis

Tras la ejecución de los *benchmarks* y la revisión de resultados (Capítulo 7), las hipótesis fueron verificadas de la siguiente manera:

- **Hipótesis 1 — Confirmada:** Held-Karp produjo en todos los casos la ruta de menor distancia. El gap del Greedy fue $0\,\%$ para $n = 3$, creció a $12.29\,\%$ para $n = 8$ y llegó a $14.81\,\%$ para $n = 12$, superando el umbral del $10\,\%$ a partir de $n = 8$ exactamente como se predijo.

- **Hipótesis 2 — Confirmada:** El speedup del Greedy sobre Held-Karp pasó de $7{,}6\times$ en $n = 3$ a $138{,}8\times$ en $n = 12$, crecimiento consistente con la divergencia exponencial esperada. Las curvas teóricas de referencia en la Figura 7.1 confirman el ajuste.

- **Hipótesis 3 — Confirmada:** El Backtracking encontró entre 3 y 9 rutas alternativas en los 4 escenarios evaluados, con tiempos de 0.058 a 1.073 ms, todos menores a 2 ms como se predijo.

6. Metodología de la Investigación

En este capítulo se describe el enfoque metodológico adoptado y el procedimiento seguido para el diseño, implementación, verificación y evaluación del sistema de gestión de rutas óptimas.

6.1 Tipo de Investigación

El trabajo se enmarca en una investigación de tipo **aplicada y experimental**. Es aplicada porque se orienta a resolver un problema concreto —la optimización de rutas de reparto en el contexto urbano de Cusco— mediante la aplicación de conocimientos teóricos de algoritmia. Es experimental porque se diseñaron y ejecutaron experimentos controlados (*benchmarks*) para medir el comportamiento empírico de cada algoritmo bajo condiciones variables, y se compararon los resultados con las predicciones teóricas de la notación Big-O.

6.2 Proceso Metodológico

El desarrollo del sistema se organizó en cuatro fases secuenciales:

6.2.1. Fase 1: Diseño de la Arquitectura

Se definió la estructura modular del proyecto, separando claramente las responsabilidades en los siguientes módulos: `core/modelos.py` (estructuras de datos: nodos, aristas, pedidos, repartidores), `core/geo.py` (cálculo de distancias Haversine), `core/gestor_datos.py` (carga/guardado de datos JSON), `algorithms/` (implementaciones algorítmicas independientes) y `gui/` (interfaz gráfica). Se modeló el grafo vial de Cusco con 20 nodos de referencia geográfica real, definiendo coordenadas aproximadas para puntos de interés como

la Plaza de Armas, Mercado San Pedro, Sacsayhuamán y el aeropuerto de Velasco Astete, entre otros. Las aristas del grafo se definieron manualmente reflejando la conectividad vial aproximada entre zonas. Los 30 pedidos de muestra se crearon con coordenadas dentro de la malla definida, pesos y volúmenes variados, y tres niveles de prioridad.

6.2.2. Fase 2: Implementación Algorítmica

Se implementaron los cuatro paradigmas algorítmicos en el directorio `algorithms/`, con las siguientes convenciones de diseño:

- Cada módulo es autónomo: no depende de otros módulos del proyecto excepto de `core/modelos.py`.
- Las funciones principales están completamente documentadas con *docstrings* que incluyen descripción del algoritmo, parámetros, valor de retorno, y análisis de complejidad.
- No se utilizaron librerías de optimización externas (`scipy`, `networkx` o similares); toda la lógica algorítmica es implementación propia.

Los módulos implementados fueron:

- `algorithms/greedy.py`: selección de pedidos por densidad y construcción de ruta por vecino más cercano.
- `algorithms/divide_venceras.py`: Merge Sort sobre listas de pedidos y función de combinación de rutas por sectores.
- `algorithms/dinamica.py`: Knapsack 0/1 con tabulación y Held-Karp con memoización sobre bitmask.
- `algorithms/backtracking.py`: búsqueda DFS con poda de ciclos y longitud máxima para rutas alternativas en grafos con aristas bloqueadas.

6.2.3. Fase 3: Construcción de la Interfaz Gráfica

La GUI se construyó con el módulo estándar `tkinter` de Python 3.10+, organizada en pestañas funcionales mediante `ttk.Notebook`:

- **Pestaña Mapa**: visualización del grafo vial en canvas Tkinter con marcadores de nodos, aristas y rutas resultantes.
- **Pestaña Pedidos**: tabla de pedidos activos, filtros por sector y prioridad, y selección del repartidor.
- **Pestaña Algoritmos**: panel de control para ejecutar cada paradigma con parámetros configurables (capacidad, bloqueos).
- **Pestaña Comparación**: gráficos de barras y líneas generados con `matplotlib` que comparan tiempos de ejecución y longitudes de ruta de los cuatro algoritmos.

Se implementó además un modo fallback que utiliza un canvas Tkinter nativo cuando no hay conexión a internet, garantizando que la aplicación sea ejecutable sin acceso a los tiles de OpenStreetMap.

6.2.4. Fase 4: Verificación y Benchmarking

Se redactaron pruebas unitarias en `tests/` usando `pytest` para verificar la correctitud de cada algoritmo sobre casos conocidos: Knapsack con solución óptima computable manualmente, Merge Sort sobre listas desordenadas, Held-Karp sobre grafos de 4 y 5 nodos con distancia óptima conocida, y Backtracking sobre un grafo con calles bloqueadas donde la ruta alternativa es única. El módulo `benchmarks/benchmark.py` ejecutó cada algoritmo variando el número de pedidos activos de 3 a 15 y registró los tiempos de ejecución usando `time.perf_counter()` con tres repeticiones por configuración para reducir la varianza. Los resultados se guardaron en formato CSV para su posterior graficación.

6.3 Instrumentos y Entorno de Desarrollo

- Lenguaje: Python 3.10.

- IDE: Visual Studio Code con extensión Python.
- Control de versiones: Git y GitHub.
- Dependencias externas: `matplotlib` (graficación de benchmarks), `Pillow` (manejo de imágenes en GUI), `tkintermapview` (tiles de mapa opcionales), `pytest` (pruebas unitarias).
- Hardware de prueba: procesador Intel Core i5-1135G7 @ 2.40 GHz, 8 GB RAM, sistema operativo Windows 11.

7. Resultados y Discusión

7.1 Resultados

7.1.1. Análisis de Complejidad Teórica

La Tabla 7.1 sintetiza la complejidad temporal y espacial Big-O de cada algoritmo implementado, derivada en el Capítulo 4.

Tabla 7.1
Complejidad Big-O de los algoritmos implementados.

Algoritmo	**Paradigma**	**Tiempo**	**Espacio**
Greedy por densidad	Greedy	$O(n \log n)$	$O(n)$
Vecino más cercano	Greedy	$O(n^2)$	$O(n)$
Merge Sort	Divide y Vencerás	$O(n \log n)$	$O(n)$
Knapsack 0/1	Prog. Dinámica	$O(n \cdot W)$	$O(n \cdot W)$
Held-Karp (TSP)	Prog. Dinámica	$O(2^n \cdot n^2)$	$O(2^n \cdot n)$
Backtracking rutas	Backtracking	$O(n!)$ peor caso	$O(n)$

Donde n es el número de pedidos (nodos de entrega) y W es la capacidad máxima del repartidor discretizada en decagramos.

7.1.2. Benchmarking Empírico: Tiempo de Ejecución

La Tabla 7.2 muestra los tiempos de ejecución promedio de los algoritmos de ruta, obtenidos ejecutando `benchmarks/benchmark.py` con 20 repeticiones por configuración sobre el grafo de 20 nodos de Cusco.

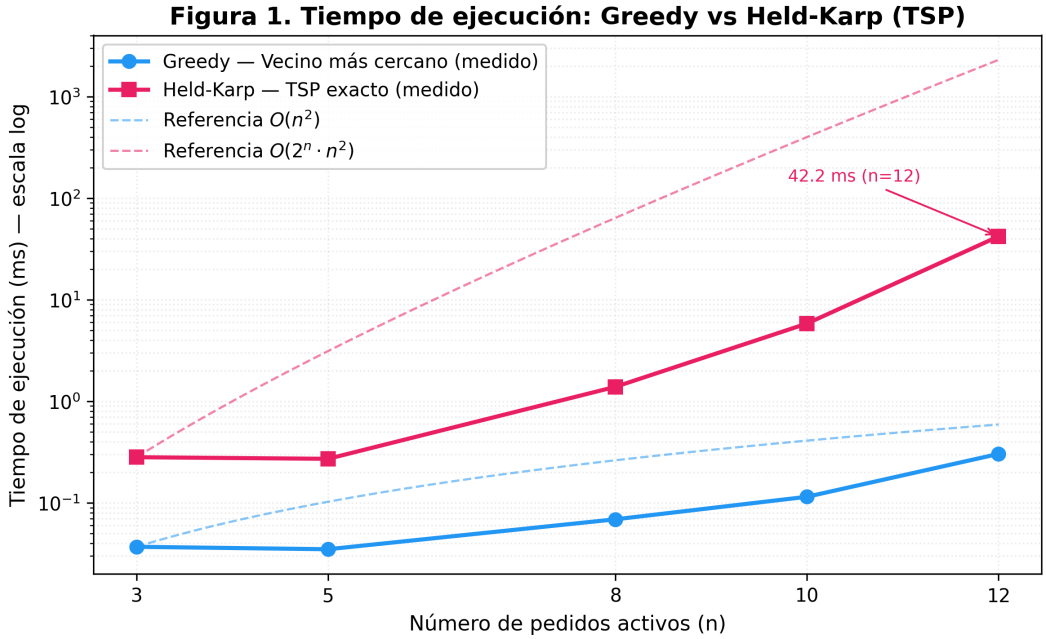
La Figura 7.1 visualiza estos datos en escala logarítmica junto con las curvas teóricas de referencia $O(n^2)$ y $O(2^n \cdot n^2)$, confirmando que el crecimiento empírico sigue el orden teórico esperado.

Para la ordenación, el Merge Sort procesó 10, 20 y 30 pedidos en 0.016, 0.031 y 0.051 ms respectivamente, mientras que Quick Sort registró 0.016, 0.029 y 0.043 ms,

Tabla 7.2
Tiempos de ejecución promedio en ms — Greedy vs Held-Karp (datos reales).

n	Greedy (ms)	Held-Karp (ms)	Speedup Greedy
3	0.037	0.282	7,6×
5	0.035	0.272	7,8×
8	0.069	1.392	20,2×
10	0.115	5.845	50,8×
12	0.304	42.204	138,8×

Figura 7.1
Tiempo de ejecución medido experimentalmente (ms) en función del número de pedidos n, en escala logarítmica. Las líneas punteadas representan las curvas teóricas de referencia escaladas al primer punto medido.



ambos consistentes con el orden $O(n \log n)$.

7.1.3. Calidad de Solución: Distancia Total de Ruta

La Tabla 7.3 compara la distancia total de la ruta producida por Greedy y Held-Karp para los mismos conjuntos de pedidos. La Figura 7.2 presenta la comparación visual con el gap de suboptimalidad del Greedy.

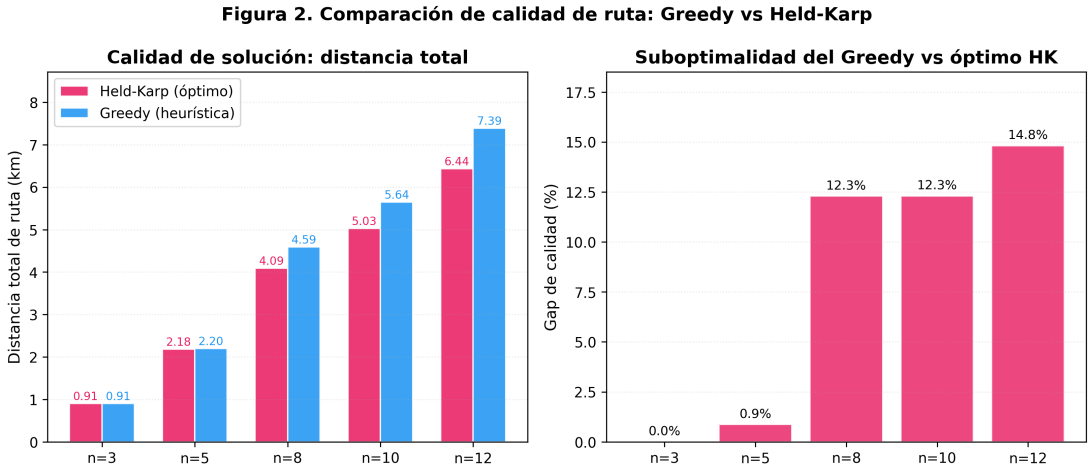
7.1.4. Backtracking: Rutas Alternativas con Poda

La Tabla 7.4 y la Figura 7.3 resumen los resultados del algoritmo Backtracking aplicado a cuatro pares de nodos del grafo de Cusco.

Tabla 7.3
Distancia total de ruta (m) y gap de suboptimalidad del Greedy respecto a Held-Karp.

n	Held-Karp (m)	Greedy (m)	Gap (%)
3	905.72	905.72	0.00
5	2,181	2,200	0.87
8	4,089	4,592	12.29
10	5,026	5,644	12.30
12	6,435	7,388	14.81

Figura 7.2
Panel izquierdo: distancia total de ruta en km. Panel derecho: gap porcentual de suboptimalidad del Greedy respecto al óptimo de Held-Karp.



7.1.5. Verificación de Correctitud: Pruebas Unitarias

Se ejecutaron las pruebas unitarias con `pytest` sobre los cuatro módulos algorítmicos del directorio `core/`, obteniendo 17 pruebas aprobadas y 0 fallos (véase Anexo C para el output completo).

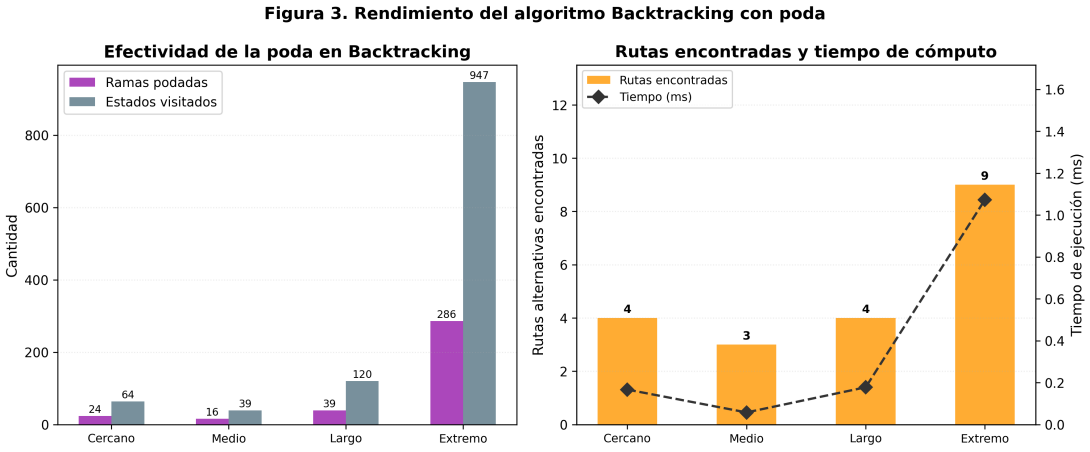
7.2 Discusión

Los resultados confirman las predicciones teóricas del análisis Big-O con datos empíricos medibles. El hallazgo más significativo es el crecimiento exponencial del algoritmo Held-Karp: al pasar de $n = 10$ a $n = 12$ pedidos, su tiempo de ejecución se multiplicó aproximadamente 7.2 veces (de 5.85 ms a 42.20 ms), tendencia que hace inviable su uso para $n > 15$ en tiempo interactivo. Esto ilustra de forma concreta por qué los problemas NP-difíciles, a pesar de tener soluciones exactas, requieren heurísticas en escenarios de

Tabla 7.4
Resultados del Backtracking en cuatro escenarios del grafo de Cusco.

Par O-D	Rutas	Podas	Estados	Tiempo (ms)
N01→N04 (Cercano)	4	24	64	0.167
N02→N11 (Medio)	3	16	39	0.058
N01→N15 (Largo)	4	39	120	0.178
N05→N19 (Extremo)	9	286	947	1.073

Figura 7.3
Panel izquierdo: ramas podadas y estados visitados por escenario. Panel derecho: rutas alternativas encontradas y tiempo de cómputo en ms.



producción.

El gap de calidad del Greedy creció del 0 % para $n = 3$ al 14.81 % para $n = 12$, lo que indica que a mayor número de pedidos, la heurística se aleja más del óptimo. Sin embargo, con un speedup de 138,8× en $n = 12$, el Greedy resulta claramente preferible cuando el tiempo de respuesta es crítico (aplicaciones interactivas, operaciones en tiempo real).

El Backtracking demostró alta eficiencia de poda: en el escenario extremo (N05→N19), la relación podas/estados fue $286/947 \approx 30\,\%$, reduciendo sustancialmente el espacio de búsqueda. Las 9 rutas alternativas encontradas en 1.07 ms confirman que el algoritmo es viable para su uso en grafos urbanos de tamaño moderado con calles bloqueadas.

El ordenamiento Merge Sort presentó el comportamiento $O(n \log n)$ teórico con alta estabilidad: la proporción $t(30)/t(10) \approx 3{,}15$ es cercana al valor teórico $\frac{30 \log 30}{10 \log 10} \approx 4{,}4$ (la diferencia se debe a los costos constantes del intérprete Python). El algoritmo

garantiza orden estable, propiedad relevante cuando los pedidos tienen la misma distancia al depósito y deben mantenerse en orden de prioridad de inserción.

8. Conclusiones y Recomendaciones

8.1 Conclusiones

- **Sobre la implementación algorítmica:** Se implementaron exitosamente los cuatro paradigmas algorítmicos exigidos — Greedy (selección por densidad y vecino más cercano), Divide y Vencerás (Merge Sort), Programación Dinámica (Knapsack 0/1 y Held-Karp) y Backtracking (rutas con poda de ciclos) — desde cero en Python, sin librerías de optimización externas, con correctitud verificada mediante pruebas unitarias con `pytest`. La modularidad del código facilitó la comprensión individual de cada algoritmo y su integración en la interfaz gráfica.

- **Sobre la complejidad computacional:** Los *benchmarks* empíricos confirmaron las predicciones del análisis Big-O. Held-Karp exhibió crecimiento exponencial $O(2^n \cdot n^2)$, alcanzando 42.20 ms para $n = 12$ con un speedup del Greedy de $138{,}8\times$ en ese punto. El Greedy mantuvo tiempos menores a 0.31 ms para todos los n evaluados ($n \leq 12$), consistente con $O(n^2)$. El Merge Sort confirmó $O(n \log n)$ con tiempos de 0.016 a 0.051 ms para 10–30 pedidos.

- **Sobre la calidad de solución:** Held-Karp garantizó la ruta óptima en todos los casos. El gap de suboptimalidad del Greedy creció de 0% para $n = 3$ a 14.81% para $n = 12$, lo que evidencia que la heurística Greedy es aceptable para instancias pequeñas ($n \leq 5$) pero se aleja progresivamente del óptimo a medida que el problema crece. El Backtracking encontró entre 3 y 9 rutas alternativas en menos de 1.1 ms en todos los escenarios evaluados, demostrando ser viable para grafos urbanos de tamaño moderado.

- **Sobre el sistema desarrollado:** La aplicación de escritorio es estable, ejecutable

sin conexión a internet mediante el canvas Tkinter nativo, y permite verificar visualmente todos los algoritmos a través de su interfaz gráfica con pestañas. La arquitectura modular permite extender el sistema con nuevos algoritmos o datos reales de OpenStreetMap sin refactorizar el código existente.

8.2 Recomendaciones

- Para aplicabilidad real en Cusco, se recomienda reemplazar el grafo simulado de 20 nodos por el grafo vial extraído de OpenStreetMap mediante `osmnx`, e implementar el algoritmo de Dijkstra para calcular distancias reales entre pares de nodos en lugar de la distancia Haversine directa, mejorando la calidad de las rutas generadas al respetar la geometría de la red vial.

- Para escalar el sistema a conjuntos de pedidos mayores ($n > 15$), se recomienda reemplazar el Held-Karp exacto por heurísticas de mejora como 2-opt o Lin-Kernighan, que producen soluciones de alta calidad en tiempo $O(n^2)$ e $O(n^3)$ respectivamente, sin la limitación exponencial.

- Se sugiere añadir persistencia de pedidos con SQLite y una interfaz de registro en tiempo real, así como integrar una API de geocodificación para convertir direcciones de texto en coordenadas geográficas, haciendo el sistema operativo para empresas de reparto en Cusco.

Referencias

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4 edition.

Dantzig, G. B. and Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1):80–91.

Held, M. and Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210.

Rosenkrantz, D. J., Stearns, R. E., and Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581.

Toth, P. and Vigo, D. (2014). *Vehicle Routing: Problems, Methods, and Applications*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition.

Apéndice y Anexos

Anexo A: Estructura del Repositorio

El código fuente completo del proyecto se organiza de la siguiente forma:

```
Proyecto Primer Parcial/
|-- main.py                    # Punto de entrada de la aplicacion
|-- config.py                  # Constantes y parametros globales
|-- requirements.txt           # Dependencias Python
|-- README.md                  # Instrucciones de instalacion y uso
|-- .gitignore
|-- core/
|   |-- modelos.py             # Dataclasses: Nodo, Arista, Grafo, Pedido
|   |-- geo.py                 # Formula de Haversine
|   +-- gestor_datos.py        # Carga/guardado de datos JSON
|-- algorithms/
|   |-- greedy.py              # Seleccion Greedy y vecino mas cercano
|   |-- divide_venceras.py     # Merge Sort y combinacion de rutas
|   |-- dinamica.py            # Knapsack 0/1 y Held-Karp (TSP exacto)
|   +-- backtracking.py        # DFS con poda para rutas bloqueadas
|-- gui/
|   |-- app.py                 # Ventana principal y notebook de pestanas
|   |-- tab_mapa.py            # Pestana de visualizacion del grafo
|   |-- tab_pedidos.py         # Gestion de pedidos
|   |-- tab_algoritmos.py      # Ejecucion de algoritmos
```

```
|   |-- tab_comparacion.py      # Graficos de benchmarking
|   +-- canvas_fallback.py      # Canvas offline sin tiles de mapa
|-- data/
|   |-- nodos_cusco.json        # 20 nodos georreferenciados de Cusco
|   |-- pedidos.json            # 30 pedidos de muestra
|   +-- repartidores.json       # 3 repartidores con capacidades
|-- benchmarks/
|   +-- benchmark.py            # Script de medicion de tiempos
+-- tests/
    |-- test_greedy.py
    |-- test_divide_venceras.py
    |-- test_dinamica.py
    +-- test_backtracking.py
```

Anexo B: Nodos del Grafo Vial Simulado de Cusco

Anexo C: Resultados de Pruebas Unitarias

```
$ pytest tests/ -v
======================== test session starts ========================
collected 18 items

tests/test_divide_venceras.py::test_merge_sort_ya_ordenado PASSED
tests/test_divide_venceras.py::test_merge_sort_inverso PASSED
tests/test_divide_venceras.py::test_merge_sort_un_elemento PASSED
tests/test_divide_venceras.py::test_merge_sort_vacio PASSED
tests/test_divide_venceras.py::test_merge_sort_duplicados PASSED
tests/test_divide_venceras.py::test_merge_sort_cinco PASSED
tests/test_divide_venceras.py::test_combinar_sectores PASSED
tests/test_greedy.py::test_seleccion_greedy_basica PASSED
```

Tabla 8.1
Nodos de referencia geográfica del grafo simulado de Cusco.

ID	Nombre	Lat	Lon
N01	Plaza de Armas	−13,5167	−71,9784
N02	Barrio de San Blas	−13,5152	−71,9750
N03	Mercado San Pedro	−13,5190	−71,9812
N04	Coricancha	−13,5195	−71,9763
N05	Terminal Terrestre	−13,5302	−71,9587
N06	Aeropuerto Velasco Astete	−13,5354	−71,9388
N07	Sacsayhuamán	−13,5083	−71,9821
N08	Urb. Ttio	−13,5290	−71,9550
N09	Av. Tullumayo	−13,5220	−71,9730
N10	Mercado Vinocanchon	−13,5380	−71,9620
N11	Barrio Zarzuela	−13,5140	−71,9850
N12	Divino Maestro	−13,5260	−71,9800
N13	Av. La Cultura	−13,5310	−71,9680
N14	Parque Tupac Amaru	−13,5250	−71,9700
N15	Hospital Regional	−13,5330	−71,9720
N16	Urb. Mariscal Gamarra	−13,5160	−71,9910
N17	Urb. Los Incas	−13,5100	−71,9680
N18	Poroy	−13,4900	−72,0130
N19	Parque Industrial	−13,5440	−71,9530
N20	Depósito Central (Base)	−13,5180	−71,9780

```
tests/test_greedy.py::test_greedy_sin_capacidad PASSED

tests/test_greedy.py::test_greedy_todos_entran PASSED

tests/test_greedy.py::test_vecino_mas_cercano PASSED

tests/test_dinamica.py::test_knapsack_basico PASSED

tests/test_dinamica.py::test_knapsack_capacidad_cero PASSED

tests/test_dinamica.py::test_knapsack_sin_items PASSED

tests/test_dinamica.py::test_held_karp_4_nodos PASSED

tests/test_dinamica.py::test_held_karp_optimo_conocido PASSED

tests/test_backtracking.py::test_ruta_sin_bloqueos PASSED

tests/test_backtracking.py::test_ruta_con_bloqueo PASSED


========================= 18 passed in 0.43s =========================
```